

Session Transcript: Five-in-a-Row with Spec Kit

A developer and an AI coding agent (Claude Code) take a small terminal game from seed documents to working, tested code using the Spec Kit workflow: constitution → specify → clarify → plan → tasks → analyze → implement.

What is shown: everything the developer typed, the agent's questions and the developer's answers, and the agent's final message for each request — for each Spec Kit command, the report it produced at the end.

What is omitted: the text of each Spec Kit command (the skill instructions the agent receives), the agent's tool calls and their output (file reads, edits, shell commands, test runs), intermediate progress notes, and session metadata. The seed documents the developer referenced (ASE-seed-development-rules.md, ASE-seed-conops.md) are not reproduced; the artifacts produced from them are in this feature folder.

Session date: 2026-10-08. **Project:** SpecKit/TicTacToe. **Feature folder:** specs/001-five-in-a-row/.

Contents

1. Constitution — /speckit-constitution
 2. Constitution (run 2) — /speckit-constitution
 3. Specification — /speckit-specify
 4. Clarification — /speckit-clarify
 5. Plan — /speckit-plan
 6. Tasks — /speckit-tasks
 7. Analysis — /speckit-analyze
 8. Analysis (run 2) — /speckit-analyze
 9. Implementation — /speckit-implement
-

1. Constitution — /speckit-constitution

Developer:

```
/speckit-constitution @../ASE-seed-development-rules.md
```

Agent:

I wrote the TicTacToe constitution from your seed rules to .specify/memory/constitution.md as **version 1.0.0**, ratified 2026-10-08. It's 1.0.0 because this is the first ratification: until now the file was only the empty template. No placeholder tokens are left, and there are no extension hooks to run because .specify/extensions.yml doesn't exist.

How your seed maps onto the template:

- **Principles I–V** follow your five sections in order: Specification and Realization; Specification Before Realization; A Person Decides Which Side Changes; Derived Documents Follow Governing Ones; Reports.
- **Rewording:** I tidied the wording ("should emphasize" became "SHOULD emphasize") and added a one-line rationale to each principle. Each "MUST" and "should" keeps the strength you gave it.
- **The template's two extra sections:**
 - **Document Hierarchy** isn't in your seed; I worked it out from Spec Kit's own file chain. It ranks the constitution, then spec.md, then the files derived from it (plan.md, research, contracts, tasks.md), then the code. Principle IV needs this ranking to say which document governs which. It also says tests count as evidence and get corrected when they contradict the spec. **Please review it.**
 - **Records of Change** puts in one place the record-keeping that Principles II and III already require. It adds nothing new except a slot for where the changelog lives.
- **Governance:** the constitution overrides other practices, and amendments follow semantic versioning. Every plan must also pass the plan template's existing "Constitution Check" and justify any deviation under "Complexity Tracking".

Open decisions for you:

- **Changelog location:** your seed requires a changelog entry when a spec changes but doesn't say where the changelog is. I left this as `TODO(CHANGELOG_LOCATION)`.
- **Principles I and V:** these keep your "should" wording, so they set an expectation, not a requirement. If you want the Constitution Check to enforce evidence and reports, they would need to become **MUST**, which is a MINOR bump to 1.1.0.
- **Sync Impact Report:** the HTML comment at the top of the file is meant for your review. Remove it before committing.

Suggested commit message:

docs: ratify constitution v1.0.0 (spec/realization principles from ASE seed rules)

2. Constitution (run 2) — /speckit-constitution

Developer:

/speckit-constitution The change log for specifications should live with the specifications.

Agent:

I amended the constitution to **version 1.0.1**: each spec's changelog now lives next to it, at `specs/<feature>/CHANGELOG.md` beside `spec.md`.

What changed in .specify/memory/constitution.md:

- **Records of Change:** the `TODO(CHANGELOG_LOCATION)` placeholder is replaced by that location.
- **Document Hierarchy:** it now says the changelog is a record of the spec's history, not a document derived from it. Without that, Principle IV ("Derived Documents Follow Governing Ones") could be read as covering the changelog.
- **Sync Impact Report:** rewritten for this amendment. As before, remove it before committing.

Why this is a patch bump (1.0.0 → 1.0.1): Principle III already required a changelog entry whenever a spec changes. This amendment only fixes where that file lives, and no principle was added, removed or redefined. If you count a new file location as materially expanded guidance, make it 1.1.0 instead.

Something I left out: I considered also requiring that a spec change and its changelog entry go in the same commit. You didn't ask for that, so it isn't in the constitution. It's worth adding if you want the history to stay easy to audit.

There are no open TODOs left.

Suggested commit message:

docs: amend constitution to v1.0.1 (spec changelog lives at `specs/<feature>/CHANGELOG.md`)

Developer: remove the sync report and commit and with the suggested commit message

Agent:

I removed the Sync Impact Report and committed the constitution with the suggested message as d58d650. The constitution file was the only thing in the commit, and the file now starts directly with `# TicTacToe Constitution`.

The diff is mostly deletions because the earlier commit, 89a8043, included the 1.0.0 Sync Impact Report, which is now gone along with the 1.0.1 one.

3. Specification — /speckit-specify

Developer:

/speckit-specify @../ASE-seed-conops.md

Agent:

I drafted the spec for the five-in-a-row game in `specs/001-five-in-a-row/spec.md`, but it has two open questions that need your answers before it's ready for planning.

The spec has three user stories:

- **P1, two players at one keyboard:** the board, turns, legal moves, win and draw. This is playable on its own.
- **P2, play against the computer:** the computer picks a random empty square.
- **P3, menus:** the start menu, “play again” and “quit”.

Beyond the stories, the spec has 17 functional requirements, edge cases for bad input, and 6 success criteria. The glossary terms your ConOps left open are fixed in the Assumptions section: “square” not “cell”, “draw” not “tie”, and “play again” for rematch.

Two other things happened alongside the spec:

- **Changelog:** I created `specs/001-five-in-a-row/CHANGELOG.md` next to the spec, as the constitution now requires, with an initial-draft entry.
- **Feature pointer:** `.specify/feature.json` now points at this feature directory, so the later Spec Kit commands can find it.

No git branch was created, because there is no branch hook installed; you're still on `main`.

The quality checklist (`checklists/requirements.md`) passes every item except “No [NEEDS CLARIFICATION] markers remain”. On the first pass I also rewrote one success criterion that wasn't measurable: the check that the computer's random moves cover every empty square now has a fixed number of trials.

Question 1: Exactly five, or five or more?

Context: FR-008 — “A player MUST win when their move makes [exactly five, or five or more?] of their marks consecutive in one line...”

What we need to know: Does a line of six or more marks count as a win?

Option	Answer	Implications
A	Five or more wins	Simplest rule and the most intuitive for new players.
B	Exactly five; six or more does not win	Classic Gomoku rule. Players must be told this, since it's surprising, and the spec needs an extra acceptance scenario for it.
Custom	Your own rule	For example, a different rule for X and O.

Question 2: Who moves first against the computer?

Context: FR-013. Your ConOps §2.2 says: “against the computer, who goes first — always me? And does that change if we play again?”

What we need to know: Does the human always play X (and so always move first), and does that change on “play again”?

Option	Answer	Implications
A	The human is always X and moves first	Simplest. The human always has the first-move advantage.
B	The human is X in the first game; sides swap on each “play again”	Fairer over a series of games. “Play again” has to track whose turn it is to start.
C	The human picks X or O before each game	Gives the player control, at the cost of one more prompt per game.
Custom	Your own rule	For example, a random choice each game.

Reply with something like “Q1: A, Q2: B”. I'll then update the spec, add a changelog entry, and re-run the checklist.

Developer: Q1: A, Q2: B

Agent:

I updated the spec with both answers, and the quality checklist now passes every item. The spec is ready for `/speckit-plan`, or for `/speckit-clarify` first if you want another round of questions.

Q1, five or more wins (FR-008):

- The rule now reads “five or more” consecutive marks in a row, a column or either diagonal.
- I added a test case to User Story 1 where X fills the gap in row 7 to make six in a row, and that counts as a win.
- The “more than five in a row” edge case now says this is a win.

Q2, sides swap on “play again” (FR-013):

- The human plays X in the first game started from the start menu. Each “play again” swaps marks, so after the human plays X the computer is X and moves first.
- I added three test cases to User Story 2: the swap from X to O, the swap back from O to X, and going back to the start menu resetting the human to X.
- The Game entity now records which mark each player holds.
- I added two assumptions you should check because I filled them in myself:
 - **Start menu resets the swap:** going back to the start menu ends the series against the computer, so the next game starts again with the human as X.
 - **No swap in two-player mode:** the two people decide between themselves who plays X, and the program doesn’t swap them on “play again”.

Records:

- **Changelog:** `CHANGELOG.md` has an entry for each decision with its reason, as Principle III requires.
- **Where the reasons came from:** the reasons recorded are the trade-offs from my option table: “simplest and most intuitive” and “fairer across a series”. If your own reasons differ, edit those lines.
- **Checklist notes:** the checklist notes record that both open questions were resolved.

Nothing is committed yet. The new files are `spec.md`, `CHANGELOG.md` and `checklists/requirements.md` in `specs/001-five-in-a-row/`, plus `.specify/feature.json`.

Developer: commit the spec files

Agent:

I committed the spec as `a978fb2` on main. The commit has the three files in `specs/001-five-in-a-row/`: `spec.md`, `CHANGELOG.md` and `checklists/requirements.md`. The commit message names both clarification decisions.

Two things were left out on purpose:

- `.specify/feature.json`: Spec Kit’s own gitignore excludes it as machine-local state.
- `.specify/scripts/python/__pycache__`: this is Python’s compiled-file cache, created when the helper scripts ran; it’s still untracked. Add it to a `.gitignore` if you want it out of `git status`.

4. Clarification — `/speckit-clarify`

Developer:

`/speckit-clarify`

Agent:

Question: Should a player be able to leave a game partway through and go back to the start menu, without closing the terminal?

Why it matters: A game on a 9-by-9 board can run up to 81 moves. Right now the spec says leaving mid-game is out of scope; I made that assumption myself, and the ConOps doesn't mention it. Without a way out, the only escape from an unwanted game is killing the program, and the acceptance tests will check for exactly that.

Recommended: Option B. Typing a word such as "quit" at any move prompt returns to the start menu with no result recorded. That's the smallest addition that gives an escape, and it fits the existing menu flow.

Option	Description
A	No. Leaving mid-game stays out of scope; the player can close the terminal (as the spec says now).
B	Yes. A command typed at any move prompt abandons the game and returns to the start menu.
C	Yes. A command typed at any move prompt ends the program entirely.
D	Yes, both: one command returns to the start menu and another quits the program.
Short	Provide a different short answer (<=5 words)

You can reply with the option letter (e.g., "A"), accept the recommendation by saying "yes" or "recommended", or provide your own short answer.

Developer: B

Agent:

I added your answer to the spec as a new requirement, FR-018: a leave-game command works at every move prompt, the prompt says how to enter it, and entering it returns to the start menu with no result. I also added a test case to User Story 3 and replaced the old "out of scope" assumption.

Question: After the computer moves, should the program also say in words which square it took (for example, "Computer plays row 4, column 7")?

Why it matters: On an 81-square grid, a single new O is easy to miss when the whole board is redrawn. A player who can't see where the computer went can't follow the game, which works against your goal that people learn by watching one game.

Recommended: Option A: name the square in words and also redraw the board. It costs one line of output and makes every computer move unmistakable.

Option	Description
A	The program names the computer's square in words and redraws the board.
B	The program only redraws the board; the player finds the new mark.
C	The program redraws the board with the computer's latest mark highlighted, without naming it in words.
Short	Provide a different short answer (<=5 words)

You can reply with the option letter (e.g., "A"), accept the recommendation by saying "yes" or "recommended", or provide your own short answer.

Developer: A

Agent:

I added FR-019: after each computer move, the program says the row and column it took and also redraws the board. User Story 2, scenario 2 now checks for this.

Question: How should a player type a square: both numbers on one line, or one number at a time?

Why it matters: The acceptance tests have to type moves in some exact form, and the "unreadable input" edge case depends on what counts as readable. The spec currently leaves this to planning, which means the planner, not you, would decide what player type.

Recommended: Option A: row then column on one line, separated by a space or a comma (4 7 or 4,7). One entry per move keeps a long game quick, and accepting either separator forgives the most common habit.

Option	Description
A	Row then column on one line, separated by a space or a comma (e.g. 4 7 or 4,7).
B	Row then column on one line, separated by a space only (e.g. 4 7).
C	Two prompts: the program asks for the row, then asks for the column.
D	Leave it to planning (as the spec says now).
Short	Provide a different short answer (<=5 words)

You can reply with the option letter (e.g., “A”), accept the recommendation by saying “yes” or “recommended”, or provide your own short answer.

Developer: A

Agent:

The clarify session is done after three questions, and I stopped early because what’s left wouldn’t change the plan or the tests. Nothing is committed yet: spec.md and CHANGELOG.md in specs/001-five-in-a-row/ have uncommitted changes.

#	Question	Your answer	Where it landed
1	Can a player leave a game partway through?	B: a leave-game command returns to the start menu with no result	New FR-018; FR-005; User Story 3 scenario 5; unreadable-input edge case; the “out of scope” assumption replaced
2	Does the program name the computer’s square in words?	A: name it and redraw the board	New FR-019; User Story 2 scenario 2
3	How does a player type a square?	A: row then column on one line, space or comma (4 7 or 4,7)	FR-004; unreadable-input edge case; the “left to planning” assumption removed

Sections touched: a new Clarifications section with today’s session, User Stories 2 and 3, Edge Cases, Functional Requirements, and Assumptions. Following the constitution, I added one entry per change to CHANGELOG.md.

Checklist: the quality checklist stays at 16/16 passing; no items changed state.

Area	Status
Scope, users, out-of-scope	Clear
Data model and game states	Clear
User journeys and error states	Resolved (questions 1 and 3)
Learnability / UX feedback	Resolved (question 2)
Performance	Clear (SC-001, SC-005)
Security, compliance, integrations	Clear (none apply: local, offline, no accounts)
Edge cases	Clear
Terminology	Clear (glossary terms fixed in Assumptions)
Acceptance testability	Clear
End of input (the terminal input stream closing, e.g. Ctrl-D)	Outstanding, low impact; planning can choose a sensible behaviour

Choices I made that you didn’t ask for:

- **Leave-game command:** FR-018 doesn’t name the command, only that the move prompt must say how to enter it. Planning will pick it unless you want to name it now (e.g. q).
- **Abandoned game and swapping sides:** after leaving mid-game, the next game against the computer starts with the human as X again. That follows from it returning to the start menu (FR-013).

Next step: /speckit-plan. Commit these changes first if you want this session kept separate in the history.

Developer: commit the spec changes

Agent:

I committed the clarified spec and its changelog as db93179 on main; the commit message lists the three decisions (FR-018, FR-019, FR-004).

The Python cache folder `.specify/scripts/python/__pycache__` is still untracked and was left out. The next step is `/speckit-plan`.

5. Plan — `/speckit-plan`

Developer:

`/speckit-plan` The plan should address the engine module, the computer opponent, and the command-line interface. Consider that future enhancements may evolve each of these independently.

Agent:

The plan is written, but one item needs your decision before `/speckit-tasks`. The spec doesn't say what happens if input ends (Ctrl-D) or the player presses Ctrl-C. Following the constitution's Principle IV, I wrote that up as a proposed spec change rather than deciding it in the plan.

The proposed rule, **P1**: "If input ends or the player interrupts the program at any prompt, the program **MUST** end promptly with a short goodbye message and without an error trace." The CLI contract includes this as rule C-17, marked as pending until you accept it. If you accept it, it goes into `spec.md` with a `CHANGELOG.md` entry; if you change or reject it, only C-17 and its tests are affected.

How the three modules stay independent:

- **Engine:** rules only, no input or output. The game state can't be changed in place; each move produces a new state. That is so a smarter opponent can try moves ahead on copies safely.
- **Opponent:** a small interface, `choose_move(state)`, with one implementation that picks uniformly among empty squares. Its random number generator can be seeded, so the 1,000-trial success criterion SC-004 gives the same result every run.
- **CLI:** the only module that reads input or prints, and the only one that remembers the mode and which mark the human holds.
- **Dependency rule:** the CLI uses the opponent and the engine, the opponent uses the engine, and nothing points back. A test will check this.
- The plan includes a table showing which single module changes for each likely enhancement: a smarter opponent, a running score, another front end, or different rules.

Decisions I made that you may want to override:

- **Language:** Python 3.12 or later with only the standard library at run time, and `pytest` run through `uv` for tests. You hadn't named a language; I chose Python because the sibling experiments in `tic-tac-toe-project/` use Python and `pytest`.
- **Leave-game command:** `m`. I avoided `q` because "quit" at the start menu ends the program.
- **Menus:** numbered, so the player types 1, 2 or 3.
- **Move entry:** a space or a comma separates row and column, and spaces around the comma are allowed (4 , 7).
- **Conformance report:** each test is tagged with the spec IDs it checks. A generated `reports/conformance.md`, not committed, lists every spec ID with its tests and flags any ID with no test.
 - Acceptance scenario IDs follow the form US1-6, derived from the spec's own numbering.
 - Edge cases aren't numbered in the spec, so their tests are tagged `EDGE-<name>` and listed separately.

The constitution check passes on all five principles both before and after the design.

All files are in `specs/001-five-in-a-row/`; no branch hook is installed, so you're still on main, and nothing is committed yet:

- plan.md
- research.md
- data-model.md
- contracts/engine.md
- contracts/opponent.md
- contracts/cli.md
- quickstart.md

Next: decide P1, then /speckit-tasks.

Developer: commit with message “SPECKIT tic tac toe - initial result of speckit-plan”

Agent:

I committed the plan as fac840c with your message. The commit has the seven plan files: plan.md, research.md, data-model.md, quickstart.md, and the three contracts under contracts/.

Two untracked items were left out:

- **ASE-seed-plan.md**: it’s now in the TicTacToe folder and I didn’t create it. I took it to be yours and didn’t open it. If it’s meant to shape the plan, tell me and I’ll check the plan against it.
- **.specify/scripts/python/__pycache__**/: Python’s cache folder, still untracked as before.

Proposal P1, how to handle end of input and Ctrl-C, still needs your decision before /speckit-tasks.

Developer: Accept proposal P1.

Agent:

P1 is now part of the spec as **FR-020**: if input ends or the player interrupts at any prompt, the program ends promptly with a short goodbye and no error trace.

Spec changes (in spec.md):

- the new requirement, FR-020;
- User Story 3, scenario 6: Ctrl-D or Ctrl-C at any menu or move prompt ends the program with a goodbye;
- an “End of input or interrupt” edge case.

Changelog: CHANGELOG.md has an entry recording that you accepted P1 and why the rule was needed: planning found the spec silent on this.

Plan documents brought into line:

- contracts/cli.md: C-17 is no longer marked pending; it now cites FR-020, and its evidence row points to FR-020 and scenario 6.
- research.md: P1 is marked accepted, with a pointer to FR-020.
- plan.md: the summary now runs to FR-020, the constitution check says P1 was accepted, and there are no open items left for you.
- data-model.md and quickstart.md: their references to “proposal P1” now point to FR-020.

Nothing is committed yet. Next step: /speckit-tasks.

Developer: commit the spec and plan changes with the message “SPECKIT tic tac toe - finalize plan”

Agent:

I committed the spec and plan changes as 6d5fbbe with your message. The commit has the seven files changed when you accepted P1: spec.md, CHANGELOG.md, plan.md, research.md, data-model.md, quickstart.md and contracts/cli.md.

Only the Python cache folder is still untracked. ASE-seed-plan.md no longer shows as untracked, so it seems you’ve moved or committed it yourself. The next step is /speckit-tasks.

Developer: What should be done with Python cache folder? Is it best to add that to .gitignore?

Agent:

Yes, add it to `.gitignore`. `__pycache__`/ holds Python’s compiled bytecode. It’s generated automatically, differs between machines and Python versions, and is rebuilt whenever it’s missing, so committing it only adds noise. It appeared when the Spec Kit helper scripts ran, and nothing ignores it yet: the repo-root `.gitignore` covers only `.DS_Store` and one Obsidian folder.

I’d put it in a new `Speckit/TicTacToe/.gitignore` rather than the repo-root one. That keeps the rules with this experiment and leaves the other experiments alone. The same file can also cover what the plan will generate once implementation starts:

```
# Python bytecode
__pycache__/
*.py[cod]

# Environments and tool caches (uv / pytest)
.venv/
.pytest_cache/

# Generated conformance report (plan research.md R7)
reports/
```

Two of these come straight from the plan rather than being general habit:

- **reports/**: the plan already says the conformance report is generated and not committed (research.md R7). The rule just makes that happen.
- **.venv/ and .pytest_cache/**: `uv run pytest` will create both.

If you want it repo-wide instead, add `__pycache__`/ and `*.py[cod]` to the root `.gitignore`. That would also cover the Python sibling experiments in `tic-tac-toe-project/`.

Should I create the project-level file and commit it?

Developer: Yes, create project-level file and commit.

Agent:

I created `Speckit/TicTacToe/.gitignore` and committed it as `d7cfec1`. `git status` is now clean for the `TicTacToe` folder; the Python cache folder no longer shows up.

The file ignores Python bytecode (`__pycache__`/, `*.pyc`), the `.venv/` and `.pytest_cache/` folders that `uv run pytest` will create, and the generated `reports/` folder.

6. Tasks — /speckit-tasks

Developer:

`/speckit-tasks`

Agent:

I wrote `specs/001-five-in-a-row/tasks.md` with 40 tasks. All 40 have the required checkbox, a sequential ID (T001–T040) and a file path; I checked the format with a script. Nothing is committed yet.

I included test tasks even though the template treats them as optional. The constitution’s Principle I requires evidence of conformance, and each contract lists the evidence it expects. Tests are written first and must fail before the code they cover is written.

Phase	Tasks	What it delivers
1 Setup	3	<code>pyproject.toml</code> , the package skeleton, a working <code>uv</code> environment

Phase	Tasks	What it delivers
2 Foundational	14	The conformance report tooling, the dependency-rule test, the whole engine, and the start menu with a clean exit on Ctrl-D/Ctrl-C
3 US1 (P1, the minimum playable version)	7	A two-player game played through to a win or draw
4 US2 (P2)	5	The random opponent and games against the computer
5 US3 (P3)	6	The game-over menu, “play again” with sides swapping, and leaving a game with m
6 Polish	5	Timing and manual-check tests, a README, the quickstart walkthroughs

How each story is checked on its own:

- **US1:** choose 2 and play scripted games where X wins, O wins, and the board fills for a draw.
- **US2:** choose 1 with a scripted fake opponent. The computer moves without reading input and names its square, and a result is announced.
- **US3:** a full session: finish a game, play again, go back to the start menu, quit. Leaving with m is tested separately.

Where I departed from the template:

- **Engine in the foundational phase:** every story plays through the engine, so I built and tested it there. The CLI start menu and the clean exit on Ctrl-D/Ctrl-C (FR-020) are there too, because every scripted CLI test starts at the menu and ends when its input runs out.
- **US2 scenarios 5–7 are finished in the US3 phase:** they test sides swapping on “play again”, and “play again” arrives with US3. The tasks and the dependency section say so.
- **US1 ends at the start menu for now:** until US3 adds the game-over menu, a finished game goes straight back to the start menu. The US1 tests stop at the result line so they won’t break when that changes.

About half the tasks can run in parallel (22 are marked). The biggest group is in the foundational phase, where seven tasks touch separate files. The opponent module can also be built alongside US1, since it depends only on the engine.

So that an agent can carry out each task without extra context:

- **Fixed test positions:** the tasks include ready-made move sequences for an X win, an O win, and a full-board draw. I checked by script that the draw board has no line longer than two.
- **Requirement tags:** every test task lists the spec IDs its tests must carry.
- **SC-002 is the one manual check:** it needs a person, so a test tagged with it skips with a “manual” reason, and the conformance report shows it as manual rather than missing.

Constitution rules written into the tasks:

- A failing test is reported to you, not “fixed” by editing the test or the spec (Principle III).
- Implementation commits say they are change type II(c).
- A spec gap is raised as a proposal, not improvised (Principle IV).

The suggested minimum to build first is Phases 1–3. Stop there and run quickstart walkthroughs M1 and M2 before continuing. The next command is `/speckit-implement`, or `/speckit-analyze` first if you want a consistency check across the spec, plan and tasks.

Developer: commit with message “SPECKIT tic tac toe - initial result of speckit-tasks”

Agent:

I committed `tasks.md` as `df82f4c` with your message; nothing else was pending in the TicTacToe folder. The next step is `/speckit-implement`, or `/speckit-analyze` first if you want a consistency check.

7. Analysis — /speckit-analyze

Developer:

/speckit-analyze

Agent:

Specification Analysis Report

The spec, plan and tasks are mostly consistent, and every requirement has at least one task. I found one critical issue and one high issue; both are one-line fixes to tasks.md. No files were changed, and no extension hooks are installed.

ID	Category	Severity	Location(s)	Summary	Recommendation
C1	Constitution	CRITICAL	tasks.md, Notes ("Commit messages")	The constitution says every commit that changes the realization MUST identify its change type (II(a), (b) or (c)). The tasks soften this to "should say so".	Change "should" to "MUST", citing Records of Change.
I1	Inconsistency	HIGH	tasks.md T015, T016, T017	T015's "input ends mid-game after 2" case needs two-player mode. T016 lets choice 2 raise <code>NotImplementedError</code> until US1. T017 then requires all tests to pass, so it can't succeed.	Move the mid-game end-of-input case into T020 (US1), or skip it until T023. Keep only the menu-level cases in T015.
C2	Constitution	MEDIUM	tasks.md T022, Notes	T022 includes "refactor T016's start menu to use <code>parse_menu</code> ". That is a change of type II(b) (structure only, behavior unchanged): Principle II requires it to be verified and its commit to state that no specified behavior changed. The Notes call every commit II(c).	Say that refactoring steps are II(b), and need a test run plus the "no specified behavior changed" statement.
U1	Underspecification	MEDIUM	contracts/cli.md §1; tasks.md T020, T031	The contract doesn't say when <code>opponent_factory</code> is called: once per session, or once per game. A scripted fake opponent used across "play again" (T031) behaves differently in each case.	Fix it in the contract, e.g. "called once per new vs-computer game", and size each scripted opponent's move list to match.
I2	Inconsistency	MEDIUM	plan.md Project Structure; tasks.md T004, T008, T020, T036, T037, T039	The tasks create files the plan's source layout doesn't list: <code>tests/conformance.py</code> , <code>tests/positions.py</code> , <code>tests/cli/conftest.py</code> , <code>tests/test_conformance.py</code> , <code>tests/test_manual_checks.py</code> , <code>tests/cli/test_timing.py</code> , <code>README.md</code> , <code>uv.lock</code> . T004 also moves the spec-ID parser out of <code>tests/conftest.py</code> , which the plan named as the plugin. This doesn't contradict the plan, but the plan no longer describes the tree.	Add these files to the layout in plan.md. Principle IV keeps the plan as the governing document for the tasks.

ID	Category	Severity	Location(s)	Summary	Recommendation
A1	Ambiguity	MEDIUM	spec.md FR-020, Edge Cases (line 148)	“End promptly” and “short goodbye message” have no measure, so a test can’t fail on either.	Give a measure, e.g. “within 1 second; a single line”, or drop “promptly”.
I3	Terminology	MEDIUM	spec.md US1, FR-001, FR-006 (“grid”) vs. Key Entities, contracts (“board”)	“Grid” and “board” are used for the same thing. The glossary in Assumptions fixes square, mark, line and draw, but not this pair.	Use “board” throughout the spec, and add it to the glossary line.
I4	Inconsistency	LOW	spec.md Key Entities (Player, Game) vs. data-model.md	The spec’s <code>Player</code> entity and <code>Game.mode</code> field don’t appear in the data model. They are split between the engine’s <code>GameState</code> and the CLI’s <code>Session</code> . That is a sound design, but the mapping isn’t written down.	Add a two-line mapping note to data-model.md.
I5	Inconsistency	LOW	tasks.md T023	Between US1 and US3, a finished game returns straight to the start menu, which doesn’t match FR-015. The tasks acknowledge this, and the conformance report will show FR-015 as having no evidence until T034.	Leave as is; mention it in the T023 commit message.
G1	Coverage	LOW	tasks.md T014, T030	SC-006 includes “bad menu choice”, but the menu-rejection tests aren’t tagged SC-006.	Add SC-006 to those tags.
U2	Underspecification	LOW	spec.md FR-012 vs. tasks.md T025	“Each equally likely” is stronger than what T025 measures (each square chosen at least once, per SC-004). Uniformity rests on the use of <code>rng.choice</code> , which only a code review can confirm.	Note in T025 that uniformity is shown by review and coverage by the test.
I6	Inconsistency	LOW	spec.md lines 3, 7; plan.md header	Both name a branch <code>001-five-in-a-row</code> that doesn’t exist; all work is on main. The spec’s Status is still “Draft”.	Write “n/a (work on main)” and set Status to “Approved”, or create the branch.
U3	Underspecification	LOW	tasks.md T006	A partial run (e.g. <code>pytest tests/engine</code>) will report every other ID as “no evidence”, which is misleading.	Have the report header say “partial run” when not all tests were collected.
U4	Underspecification	LOW	spec.md SC-001; tasks.md T036	SC-001 measures a person’s time to first move. T036 measures only the program’s start-up time, so it is a partial stand-in.	Note in T036 that the human part is checked in quickstart M4.
U5	Constraint	LOW	plan.md Constraints (“offline”); tasks.md T001	The first <code>uv sync</code> downloads <code>hatchling</code> and <code>pytest</code> . Only running the game is offline.	Change the constraint to “offline at run time”.
D1	Duplication	LOW	spec.md FR-005/SC-006, FR-017/edge case “Invalid menu choice”, US2-3/SC-004	Overlapping statements, each pairing a rule with its measure. They don’t conflict.	No change needed.
S1	Style	LOW	spec.md line 139	One line is over 100 characters, left over from the clarify edit.	Re-wrap it.

Coverage Summary

Requirement	Has task?	Task IDs	Notes
FR-001	✓	T009, T010, T011	
FR-002	✓	T009, T010, T020	
FR-003	✓	T010, T012, T020	
FR-004	✓	T009, T018, T019, T022	
FR-005	✓	T010, T020, T021	
FR-006	✓	T018, T020, T021	
FR-007	✓	T020, T021, T032	
FR-008	✓	T010, T012	
FR-009	✓	T010, T020, T026	
FR-010	✓	T010, T020	
FR-011	✓	T020, T028	
FR-012	✓	T025, T026, T027	U2
FR-013	✓	T026, T031, T032, T034	U1
FR-014	✓	T014, T016	
FR-015	✓	T030, T034	I5
FR-016	✓	T014, T030	
FR-017	✓	T014, T030	
FR-018	✓	T032, T033, T034	
FR-019	✓	T026, T028	
FR-020	✓	T015, T016	I1, A1
SC-001	✓	T036	U4
SC-002	✓ (manual)	T037, T040	
SC-003	✓	T010, T020	
SC-004	✓	T025	
SC-005	✓	T026	
SC-006	✓	T010, T020	G1

All 20 acceptance scenarios and 8 edge cases also have tagged tests.

Constitution Alignment

- **C1:** the tasks weaken a MUST in Principle II / Records of Change to “should”.
- **C2:** a II(b) step is mislabelled as II(c).
- **Principles III, IV and V:** reflected correctly. A failing test goes to you to decide; spec gaps are raised as proposals; the conformance report is generated by the tests.

Tasks not tied to a spec requirement None of these is a problem:

- **T001–T003:** setup.
- **T004–T008:** evidence tooling, the dependency-rule test, and shared test positions. They trace to the constitution and plan.md R3/R7 rather than to the spec.
- **T039:** the README.

Metrics

- **Requirements:** 26 (20 FR, 6 SC).
- **Tasks:** 40.
- **Coverage:** 100% of requirements have at least one task.
- **Ambiguities:** 1. **Duplications:** 1, grouping three overlaps.
- **Critical issues:** 1.

Next Actions

- **Fix C1 before /speckit-implement.** It's a one-word change in tasks.md.

- **Fix I1 too.** Otherwise the foundational checkpoint (T017) cannot pass as written.
- **Change the documents in this order:**
 - **Spec** (A1, I3, I6, S1): each change needs a CHANGELOG.md entry.
 - **Plan and contracts** (U1, I2, U5).
 - **Tasks** (C1, C2, I1, G1, U2, U3, U4).
- **The LOW items can wait**, or be fixed in one cleanup pass.

Would you like me to suggest concrete edits for the top 7 issues (C1, I1, C2, U1, I2, A1, I3)?

Developer: Create a markdown report reflecting the results of this analysis. Then, follow your suggestion. Tell me what to do in detail: suggest concrete edits for what should be done before proceeding with implementation.

Agent:

I saved the analysis as `specs/001-five-in-a-row/analysis-report.md`. Part 1 has the findings, the coverage table and the metrics. Part 2 has eight edits, E1–E8, each with exact before-and-after text. I checked that each “replace this” passage in `tasks.md` matches the current file exactly. Nothing else was changed, and the report itself isn’t committed.

Changes that need your decision (to `spec.md` and the CLI contract):

1. **E6, FR-020 needs a measure.** “End promptly with a short goodbye message” can’t fail a test. The proposed wording is “MUST print a single goodbye line and end within 1 second, without an error trace”. The edge case and contract rule C-17 change to match. Choose a different limit if 1 second isn’t what you want.
2. **E7, use “board” everywhere.** The spec says “grid” in 11 places, while the contracts say “board”. The edit changes them all, rewrites FR-001 and the Board entity line, and adds “board (not grid)” to the glossary. The glossary would also say “draw the board” means display it, since “draw” already names the result.
3. **E4, when the computer opponent is created.** The contract doesn’t say how often the program creates the computer opponent. The proposal: once each time “Play the computer” is chosen, kept for every “play again” in that series. That matches how the plan already tracks a series of games. Creating a fresh opponent for every game also satisfies the spec; pick it if you prefer.

E6 and E7 change the spec, so they need a CHANGELOG.md entry; the report includes suggested text.

Mechanical fixes (to `tasks.md` and `plan.md`):

4. **E1 (the critical finding):** the commit-message note says “should”; change it to “MUST”. The replacement text also explains when a commit is II(a), II(b) or II(c).
5. **E2 (the high finding):** T015’s “input ends mid-game” test can’t pass before two-player mode exists. The edit moves that case into T020 and keeps only the start-menu cases in T015.
6. **E3:** label T022’s start-menu refactor as a separate II(b) commit, verified by a full test run.
7. **E5:** update the plan’s source layout to list the eight files the tasks create, such as `tests/conformance.py`, `tests/positions.py` and `uv.lock`.
8. **E8:** add the SC-006 tag to the two invalid-menu-choice tests (T014, T030).

Order to apply them:

1. The spec and changelog (E6, E7).
2. The CLI contract and plan (E4, E5).
3. `tasks.md` (E1, E2, E3, E8).
4. Re-run `/speckit-analyze`. Only low-severity items should remain.
5. Then `/speckit-implement`.

The other low-severity items have one-line fixes in a table at the end of the report; they can wait until before T038.

Should I apply E1–E8 as written, or with changes to the three decisions above?

Developer: Apply the changes as written, then commit with message “SPECKIT tic tac toe - updates to specs, plan, tasks after first speckit-analyze”

Agent:

I applied edits E1–E8 as written and committed them as `ba48fd3` with your message. I also included `analysis-report.md` in the commit, since it wasn’t committed before.

- **spec.md:**
 - **FR-020 and its edge case:** now “a single goodbye line, within 1 second” (E6).
 - **“Board” for “grid”:** used in every place, FR-001 and the Board entity line rewritten, and the glossary extended (E7). No “grid” remains except in “(not grid)”.
 - **Long line:** the one over 100 characters is re-wrapped (S1).
- **CHANGELOG.md:** an entry for both spec changes, with their reasons.
- **contracts/cli.md:** says when the computer opponent is created, once per series of “play again” games (E4), and C-17 now includes the 1-second limit (E6).
- **plan.md:** the source layout lists the eight files the tasks create (E5).
- **tasks.md:**
 - **Commit-message note:** now says MUST and explains II(a), (b) and (c) (E1).
 - **End-of-input tests:** the mid-game case moved from T015 to T020, so the foundational checkpoint can pass (E2).
 - **T022 refactor:** marked as a separate II(b) commit (E3).
 - **Scripted fake opponent:** T020 says one instance covers a whole series (E4).
 - **1-second limit:** T015 checks it (E6).
 - **SC-006 tag:** added to T014 and T030 (E8).

The next step is to re-run `/speckit-analyze`, which should find only the low-severity items left, then `/speckit-implement`.

8. Analysis (run 2) — `/speckit-analyze`

Developer:

`/speckit-analyze`

Agent:

Specification Analysis Report (second run)

The edits from the first run worked. The critical, high and medium findings from that run are all resolved, and every requirement still has a task. This run found one new medium issue, which the first round of edits missed, plus some low items. Nothing blocks `/speckit-implement`. No files were changed, and no extension hooks are installed.

Status of the first run’s findings

ID	First run	Now	Evidence
C1	CRITICAL	Resolved	The tasks.md commit-message note says MUST and defines II(a), (b) and (c).
I1	HIGH	Resolved	T015 tests only the start menu; the mid-game case is in T020, so T017’s “all pass” checkpoint is now reachable.
C2	MEDIUM	Resolved	T022’s refactor is a separate II(b) commit, with the full test suite run before it.
U1	MEDIUM	Resolved	contracts/cli.md §1 says when <code>opponent_factory</code> is called; T020’s fake opponent matches.
I2	MEDIUM	Resolved	The plan’s source layout lists every file the tasks create.
A1	MEDIUM	Resolved, with one place missed	FR-020, the edge case and C-17 now say “single goodbye line, within 1 second”. See I7.
I3	MEDIUM	Resolved in the spec	No “grid” left in spec.md except the glossary’s “(not grid)”. See I9.
G1	LOW	Resolved	T014 and T030 are tagged SC-006.
S1	LOW	Resolved	No line in spec.md is over 100 characters.

ID	First run	Now	Evidence
I4, I5, U2, U3, U4, U5, I6, D1	LOW	Open (not part of the edits)	Unchanged; see below.

Findings

ID	Category	Severity	Location(s)	Summary	Recommendation
I7	Inconsistency	MEDIUM	spec.md L127–129 (US3 scenario 6)	The acceptance scenario still says “prints a short goodbye”, while FR-020 and the edge case now say “a single goodbye line ... within 1 second”. The first-round edits (E6) missed this line.	Change it to “prints a single goodbye line and ends within 1 second, without an error trace”, with a matching CHANGELOG line.
I8	Inconsistency	LOW	research.md L146–147 (P1)	Quotes P1’s original wording (“end promptly with a short goodbye message”). It is a record of the proposal, but it no longer matches the spec.	Add to P1’s status line: “Wording since tightened; see FR-020 and CHANGELOG.”
I9	Terminology	LOW	data-model.md L35	The Board’s <code>cells</code> field is described as a “9×9 grid”. The spec now uses “board” only.	Change to “9×9 arrangement of squares, each a Mark or empty”.
I4	Inconsistency	LOW	spec.md Key Entities (Player, Game) vs. data-model.md	Open from the first run: the spec’s <code>Player</code> and <code>Game.mode</code> are split between <code>GameState</code> and <code>Session</code> , and the mapping isn’t written down.	Add the mapping note (analysis-report.md, LOW table).
I5	Inconsistency	LOW	tasks.md T023	Open from the first run: until US3, a finished game returns straight to the start menu. This is acknowledged in the task.	Mention it in T023’s commit message.
U2	Underspecified	LOW	spec.md FR-012; tasks.md T025	Open from the first run: “each equally likely” is shown by code review, not by the test.	Add a note to T025.
U3	Underspecified	LOW	tasks.md T006	Open from the first run: a partial test run reports every other ID as “no evidence”.	Add a “partial run” header.
U4	Underspecified	LOW	spec.md SC-001; tasks.md T036	Open from the first run: T036 measures only the program’s start-up part of SC-001.	Point to quickstart M4 for the human part.
U5	Constraint	LOW	plan.md L47	Open from the first run: “offline” holds only at run time, not for the first <code>uv sync</code> .	Reword.
I6	Inconsistency	LOW	spec.md L3, L7; plan.md L3	Open from the first run: both name branch <code>001-five-in-a-row</code> , which doesn’t exist; spec Status is still “Draft”.	Write “n/a (work on main)”; set Status when you approve the spec.
D1	Duplication	LOW	spec.md FR-005/SC-006; FR-017/edge case; US2-3/SC-004	Overlapping rule-and-measure pairs; they don’t conflict.	No change.

Coverage This is unchanged from the first run’s table in analysis-report.md, apart from the new tags:

- **Requirements:** all 20 FRs and 6 SCs have at least one task. SC-006 now also has T014 and T030; FR-020 now has T015, T016 and T020.
- **Scenarios and edge cases:** all 20 acceptance scenarios and 8 edge cases have tagged tests.
- **Spec-ID check:** the edge-case names are unchanged, so T005's fixed list of expected IDs still matches the spec.

Constitution alignment No issues. Principle II and the Records of Change are now stated correctly in tasks.md. Principles I, III, IV and V are reflected as before. Both spec changes from the first round have CHANGELOG entries.

Tasks not tied to a spec requirement As before, all expected: T001–T008 (setup and evidence tooling) and T039 (the README).

Metrics

Metric	First run	Now
Requirements (FR + SC)	26	26
Tasks	40	40
Coverage	100 %	100 %
Ambiguities	1	0
Duplications	1	1
Critical / High	1 / 1	0 / 0
Medium	5	1
Low	10	10

Next Actions

- **You can go ahead with /speckit-implement:** nothing is critical or high.
- **Fix I7 first anyway.** It is a two-line spec change plus a CHANGELOG line. Otherwise the US3 scenario 6 test would rest on the vaguer wording.
- **I8 and I9 are one-line touch-ups** to research.md and data-model.md, so they need no changelog entry.
- **The remaining low items can wait until T038**, the final conformance check. The one-line fixes are already in analysis-report.md Part 2, under “LOW items”.

Would you like me to suggest concrete edits for I7, I8 and I9?

Developer: Apply edits for I7, I8, I9 and commit

Agent:

I applied the three edits and committed them as 951832b. You didn't give a message this time, so I wrote one: “SPECKIT tic tac toe - fixes from second speckit-analyze (I7, I8, I9)”.

- **I7, spec.md:** User Story 3 scenario 6 now says “prints a single goodbye line and ends within 1 second, without an error trace”, matching FR-020. Because this changes the spec, I added a CHANGELOG.md entry noting that the first round of edits missed it.
- **I8, research.md:** proposal P1's status line now says the wording was later tightened, pointing to FR-020 and the changelog.
- **I9, data-model.md:** the Board's cells field is now a “9×9 arrangement of squares, each a Mark or empty”, not a “grid”.

Only low-severity items remain, and none of them blocks /speckit-implement. Their one-line fixes are in analysis-report.md if you want them before the final conformance check (T038).

9. Implementation — /speckit-implement

Developer:

/speckit-implement

Agent (question): tasks.md says to commit after each task or logical group, with each message stating its change type (II(b)/II(c)), and makes T022's refactor a separate II(b) commit. Should I make those commits as I implement?

- **Commit per phase (Recommended)** — I commit at each phase checkpoint (and T022's refactor separately), each message ending with its Change type line. You can review/squash afterwards.
- **Don't commit** — Leave everything uncommitted; I give you suggested commit messages at the end. T022's refactor can't then be a separate commit.
- **Commit per task** — One commit per task (about 40 commits), each with its change type.

Developer (answer): Commit per phase (Recommended)

Agent:

All 40 tasks are done and checked off in tasks.md. The game is playable with `uv run five-in-a-row`, and all 102 automated tests pass. The conformance report (reports/conformance.md) shows 53 of the 54 spec IDs passing and none failing or missing a test. The remaining one, SC-002, is marked as a manual check.

I committed once per phase, plus the separate refactor commit T022 required. Each message ends with its Change type: line:

Commit	Phase	Change type
3b9a45d	1 Setup (T001–T003)	II(c)
2ff1e1c	2 Foundational: report tooling, dependency-rule test, engine, CLI shell (T004–T017)	II(c)
5f105bc	3 US1, two players (T018–T024; T022's refactor in the next commit)	II(c)
c79996b	T022 start-menu refactor	II(b), full test suite run before committing
8fcbe1d	4 US2, play the computer (T025–T029)	II(c)
ff29322	5 US3, game-over menu, play again, leave a game (T030–T035)	II(c)
ea0decc	6 Polish (T036–T040)	II(c)

What still needs you:

- **SC-002 needs a person:** someone who has watched one game should play one without help.
- **T040's walkthroughs were scripted:** I ran quickstart M1–M5, M7 and M8 against the real program with piped input, and sent a real interrupt signal for Ctrl-C. All matched. A person hasn't played it, and M6 (going back to the start menu resets you to X) is covered only by tests.

Choices the documents didn't spell out:

- **Input is echoed when stdin isn't a terminal:** this makes transcripts read like the screen; it changes nothing at a real terminal. The CLI contract doesn't mention it. Principle IV says such additions belong in the contract, so I can add a line to contracts/cli.md if you agree.
- **One small public addition to the engine:** `Square.on_board()`, which isn't in the engine contract.
- **I wrote the exact on-screen wording:** for example "Row 5, column 5 is occupied." and "Computer (O) plays row 4, column 7.". The contract only requires what each message must say.
- **Phase 1's commit is labelled II(c)** even though it adds no behavior; it was setup only.

Bugs found: no test failed after its code was written, so there were no spec-versus-code disagreements to bring to you. Only one test of my own needed fixing: the win-on-the-last-square board, which I rebuilt from a board checked by script before the engine existed.

The low-severity analysis items are still open (listed in analysis-report.md). Nothing is uncommitted except the generated reports/ folder, which is git-ignored. No extension hooks are installed.